

CAMPUSGUARD

TEAM MEMBERS:

Kayla Falconer – u25006747

Vashti Pillay – u25041887

Lindokuhle Skosana – u24891968

TASK 1

Overview

CampusGuard is an emergency response software system that allows university campuses to manage on site facilities and staff members for quick and reliable responses to emergencies and minor everyday incidents, while also allowing for coordination with government emergency services for more intense and serious incidents that endanger the lives and health of students and staff members. Our system has integrated six GoF design patterns to achieve this goal.

Design Patterns

FAÇADE

REASONING

- The EmergencyResponseFacade is responsible for delegating the different functions associated with each underlying subsystem and reducing the information needed by the client to utilize the software therefore it is important that the EmergencyResponseFacade also deletes the attributes it owns

PARTICIPANTS

Façade	EmergencyResponseFacade
Subsystem classes	Dispatcher, IncidentControl, CommunicationHub, SecurityGuards, FirstAidTeam, FacilityStaff, AccessControlTeam, AmbulanceAdapter, PoliceAdapter, FireFighterAdapter, IncidentHistory

OWNERSHIP SUMMARY

Object	Owner	Deleted in
CommunicationHub	EmergencyResponseFacade	~EmergencyResponseFacade()
SecurityGuards, FirstAidTeam, FacilityStaff, AccessControlTeam	EmergencyResponseFacade	~EmergencyResponseFacade()
AmbulanceAdapter, PoliceAdapter, FireFighterAdapter	EmergencyResponseFacade	~EmergencyResponseFacade()
Ambulance	AmbulanceAdapter	~AmbulanceAdapter()
Police	PoliceAdapter	~PoliceAdapter()
FireFighter	FireFighterAdapter	~FireFighterAdapter()
Dispatcher	EmergencyResponseFacade	~EmergencyResponseFacade()
IncidentControl	EmergencyResponseFacade	~EmergencyResponseFacade()
IncidentHistory	EmergencyResponseFacade	~EmergencyResponseFacade()
Protocol* commands	Dispatcher	undoLast() or ~Dispatcher()
IncidentState*	IncidentControl	setState() or ~IncidentControl()
IncidentMemento*	IncidentHistory	Transferred to caller on pop(); caller must delete after

Subsystem Attributes Owned by Facade

Attribute Name	Type	Owned?	Purpose
dispatcher	Dispatcher*	YES	Internal command invoker and history manager

Attribute Name	Type	Owned?	Purpose
controller	IncidentControl*	YES	State machine context and memento originator
hub	CommunicationHub*	YES	Internal mediator hub for colleague broadcasts
guards	SecurityGuards*	YES	Security team colleague
medics	FirstAidTeam*	YES	Medical team colleague
facility	FacilityStaff*	YES	Facility staff colleague
access	AccessControlTeam*	YES	Access control team colleague
police	EmergencyResponder*	YES	Points to PoliceAdapter
ambulance	EmergencyResponder*	YES	Points to AmbulanceAdapter
fire	EmergencyResponder*	YES	Points to FireFighterAdapter
history	IncidentHistory*	YES	Memento caretaker history stack

Facade Workflow Subsystem Coordination

reportIncident()

Step	Subsystem	Action Taken	Pattern(s)
1	IncidentControl	controller->addThreat(location, threat) registers threat and escalates to Urgent	State
2	IncidentHistory	history->push(controller->createMemento()) saves post-threat snapshot	Memento
3	CommunicationHub	hub->notify(nullptr, "INCIDENT_REPORTED") broadcasts event to all colleagues	Mediator
4	Dispatcher	dispatcher->issueCommand(new Isolate(access, location)) isolates zone	Command

escalateToEmergency()

Step	Subsystem	Action Taken	Pattern(s)
1	IncidentControl	controller->escalate() transitions context to Emergency state	State
2	IncidentHistory	history->push(controller->createMemento()) saves snapshot with label "EMERGENCY"	Memento
3	Dispatcher	Issues EmergencyEscalation command calling respond() on adapters	Command + Adapter
4	CommunicationHub	hub->notify(nullptr, "EMERGENCY_DECLARED") triggers emergency reactions across all teams	Mediator

resolveIncident()

Step	Subsystem	Action Taken	Pattern(s)
1	Dispatcher	dispatcher->issueCommand(new Resolve(facility, access)) restores access and dispatches maintenance	Command
2	IncidentControl	controller->deescalate() transitions state to Resolved	State
3	IncidentHistory	history->push(controller->createMemento()) saves post-resolve snapshot	Memento
4	CommunicationHub	hub->notify(nullptr, "INCIDENT_RESOLVED") stands down security and restores normal rules	Mediator

rollbackLastAction()

Step	Subsystem	Action Taken	Pattern(s)
1	IncidentHistory	history->pop() removes and returns top IncidentMemento*	Memento

Step	Subsystem	Action Taken	Pattern(s)
2	IncidentControl	controller->restore(memento) reconstructs previous threat count, threats, and state	Memento + State
3	Dispatcher	dispatcher->undoLast() calls undo() on last command and deletes it	Command

COMMAND

Participants

Role	Class(es)
Command Interface	Protocol
Invoker	Dispatcher
ConcreteCommands	Evacuate, Deescalate, EmergencyEscalation, Resolve, Assist, GrantAccess, Isolate
Receivers	SecurityGuards, FirstAidTeam, FacilityStaff, AccessControlTeam, EmergencyResponder

Protocol Interface Methods

Method	Virtual?	Return Type	Purpose
execute()	Pure virtual	void	Executes the concrete command operation on its receiver entities
undo()	Pure virtual	void	Reverses the effects of the previously executed command on its receivers
~Protocol()	Virtual (not pure)	void	Ensures safe polymorphic destruction of concrete command objects

Dispatcher Class Members

Name	Type / Signature	Access	Purpose
commandQueue	std::queue<Protocol*>	Private	Stores pending commands scheduled for future execution
commandHistory	std::stack<Protocol*>	Private	Stores executed commands for sequential undo operations
issueCommand()	issueCommand(c : Protocol*) : void	Public	Executes c->execute() and pushes onto commandHistory
undoLast()	undoLast() : void	Public	Pops last c, calls c->undo(), and deletes c

Concrete Commands Receiver Mapping

Command	Associated Receivers	Receiver Ownership
Evacuate	SecurityGuards*, FacilityStaff*, AccessControlTeam*	Non-owning aggregate pointers
Deescalate	SecurityGuards*	Non-owning aggregate pointer
EmergencyEscalation	SecurityGuards*, FacilityStaff*, AccessControlTeam*, FirstAidTeam*, EmergencyResponder* (x3)	Non-owning aggregate pointers
Resolve	FacilityStaff*, AccessControlTeam*	Non-owning aggregate pointers
Assist	FirstAidTeam*, AccessControlTeam*	Non-owning aggregate pointers
GrantAccess	AccessControlTeam*	Non-owning aggregate pointer
Isolate	AccessControlTeam*	Non-owning aggregate pointer

MEDIATOR

Participants

Role	Class(es)
Mediator Interface	CommunicationTeam
ConcreteMediator	CommunicationHub
Colleague (Abstract)	FirstResponder
ConcreteColleagues	SecurityGuards, FirstAidTeam, FacilityStaff, AccessControlTeam

Colleague Event Reactions

Colleague	Event String	Reaction Taken
SecurityGuards	"INCIDENT_REPORTED"	Logs security awareness; increases patrol vigilance
SecurityGuards	"EMERGENCY_DECLARED"	Triggers immediate perimeter security lockdown
SecurityGuards	"INCIDENT_RESOLVED"	Stands down active tactical teams
SecurityGuards	"ZONE_LOCKED"	Deploys guards to monitor locked zone boundary
FirstAidTeam	"INCIDENT_REPORTED"	Prepares medical equipment and triage kits
FirstAidTeam	"BACKUP_NEEDED"	Dispatches additional medical personnel to requested sector
FirstAidTeam	"EMERGENCY_DECLARED"	Prepares field trauma unit for incoming casualties
FacilityStaff	"BUILDING_CLEARED"	Locks down utility infrastructure
FacilityStaff	"INCIDENT_RESOLVED"	Calls dispatchMaintenance() to inspect damage
FacilityStaff	"ZONE_LOCKED"	Secures secondary utility access points within locked sector
AccessControlTeam	"EMERGENCY_DECLARED"	Unlocks all evacuation turnstiles and emergency exits
AccessControlTeam	"INCIDENT_RESOLVED"	Restores normal card-access security rules

ADAPTER

Participants

Role	Class(es)
Target Interface	EmergencyResponder
Adapters	AmbulanceAdapter, PoliceAdapter, FireFighterAdapter
Adaptees	Ambulance, Police, FireFighter
Client	EmergencyEscalation (Command)

Interface Mismatch Translation Summary

Service	CampusGuard Call	Adaptee Native Method	Adapter Translation Logic
Ambulance	respond(location, threat)	dispatch(caseType, lat, lon)	Maps location string to GPS lat/lon doubles; maps Threat enum to caseType integer.

Service	CampusGuard Call	Adaptee Native Method	Adapter Translation Logic
Police	respond(location, threat)	dispatch(location, severity, incidentCode)	Maps Threat enum to numeric severity level and radio incidentCode string.
FireFighter	respond(location, threat)	alertStation(location, buildingNumber)	Maps location string to integer buildingNumber.

STATE

REASONING

- The State pattern was chosen for Incidents to better track and alter the behaviour of an Incident for a given scenario because each Incident requires a different set of operations and actions to be performed based on the context the best choice for the Incident subsystem was State

Participants

Role	Class(es)
State Interface	IncidentState
ConcreteStates	Moderate, Urgent, Emergency, Resolved
Context	IncidentControl

State Transition Matrix

Current State	escalate() Action	deescalate() Action	getLabel() Return
Moderate	context->setState(new Urgent())	If threatCount == 0, transitions to Resolved; else logs warning	"MODERATE"
Urgent	context->setState(new Emergency())	context->setState(new Moderate())	"URGENT"
Emergency	INVALID: Prints warning; no state transition	context->setState(new Urgent())	"EMERGENCY"
Resolved	context->setState(new Moderate()) (Re-opens incident)	INVALID: Prints warning; no state transition	"RESOLVED"

MEMENTO

REASONING

- The Memento pattern is utilized to work in conjunction with the State and Command pattern to allow the undo operation to be reflected by the IncidentState as well instead of creating a brand new IncidentState the previous state is popped from the IncidentHistory class, this ensures that the client code is decoupled from the intricate logic associated with undo commands as real life emergency situations often have plenty mistakes.

Participants

Role	Class(es)
Originator	IncidentControl
Memento	IncidentMemento
Caretaker	IncidentHistory

IncidentMemento Attributes

Attribute Name	Type	Access	Reason for Value Copy
threatCount	int	Private	Primitive integer snapshot; no pointers/aliasing.
activeThreats	std::map<std::string, Threat>	Private	Value-copy map snapshot independent of originator.
stateLabel	std::string	Private	Avoids dangling IncidentState* pointers when states change.

SYSTEM-WIDE OWNERSHIP SUMMARY

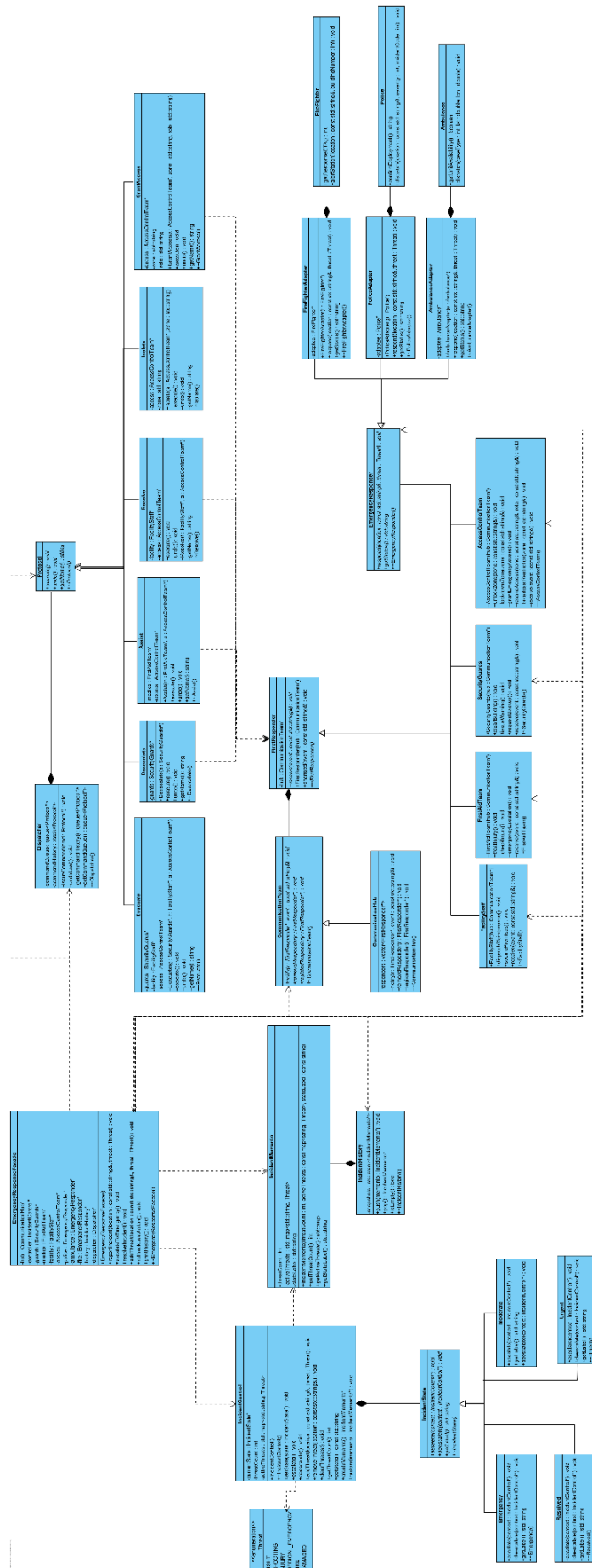
Complete Ownership & Lifetime Decisions

Object	Owner	Deleted in
CommunicationHub	EmergencyResponseFacade	~EmergencyResponseFacade()
SecurityGuards, FirstAidTeam, FacilityStaff, AccessControlTeam	EmergencyResponseFacade	~EmergencyResponseFacade()
AmbulanceAdapter, PoliceAdapter, FireFighterAdapter	EmergencyResponseFacade	~EmergencyResponseFacade()
Ambulance	AmbulanceAdapter	~AmbulanceAdapter()
Police	PoliceAdapter	~PoliceAdapter()
FireFighter	FireFighterAdapter	~FireFighterAdapter()
Dispatcher	EmergencyResponseFacade	~EmergencyResponseFacade()
IncidentControl	EmergencyResponseFacade	~EmergencyResponseFacade()
IncidentHistory	EmergencyResponseFacade	~EmergencyResponseFacade()
Protocol* commands	Dispatcher	undoLast() or ~Dispatcher()
IncidentState*	IncidentControl	setState() or ~IncidentControl()
IncidentMemento*	IncidentHistory while stored	Transferred to caller on pop(); caller deletes post-restore().

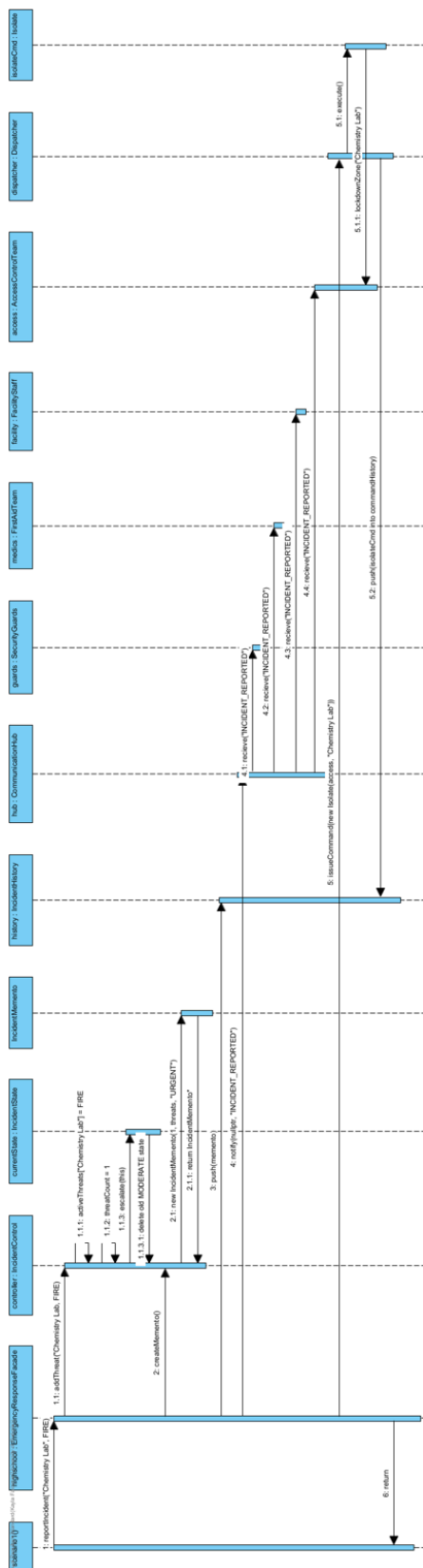
Non-Owning References (Never Deleted)

Class	Pointer Held	Why It Is Non-Owning
CommunicationHub	FirstResponder* in responders vector	Responders are owned and destroyed by EmergencyResponseFacade.
Concrete commands	Receiver pointers (guards*, access*, etc.)	Receivers are owned by Facade; commands only hold references.
FirstResponder	hub*	The hub is owned by EmergencyResponseFacade.

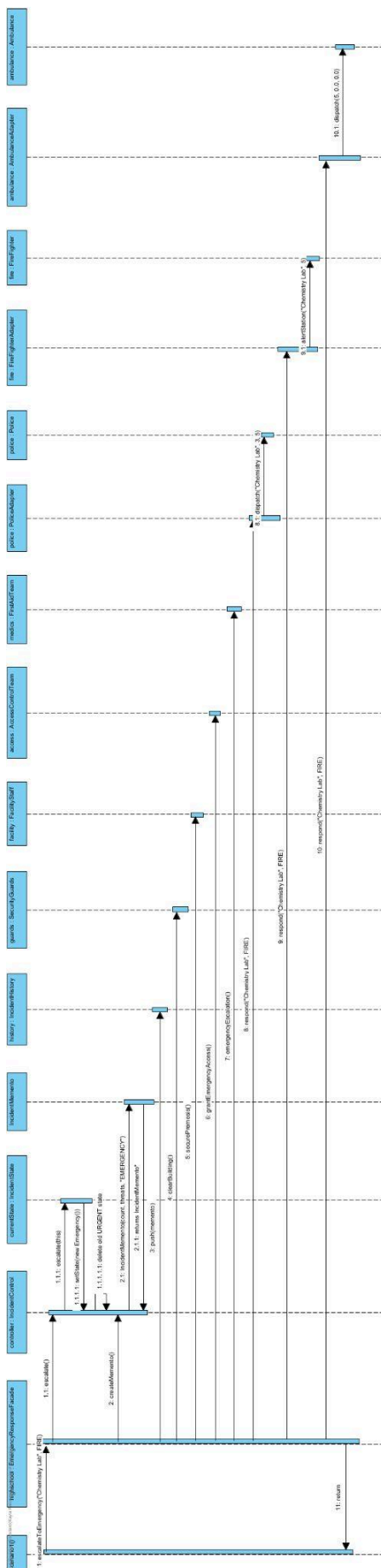
UML DIAGRAMS



Sequence Diagram that includes Command and at least one other required pattern (Facade)

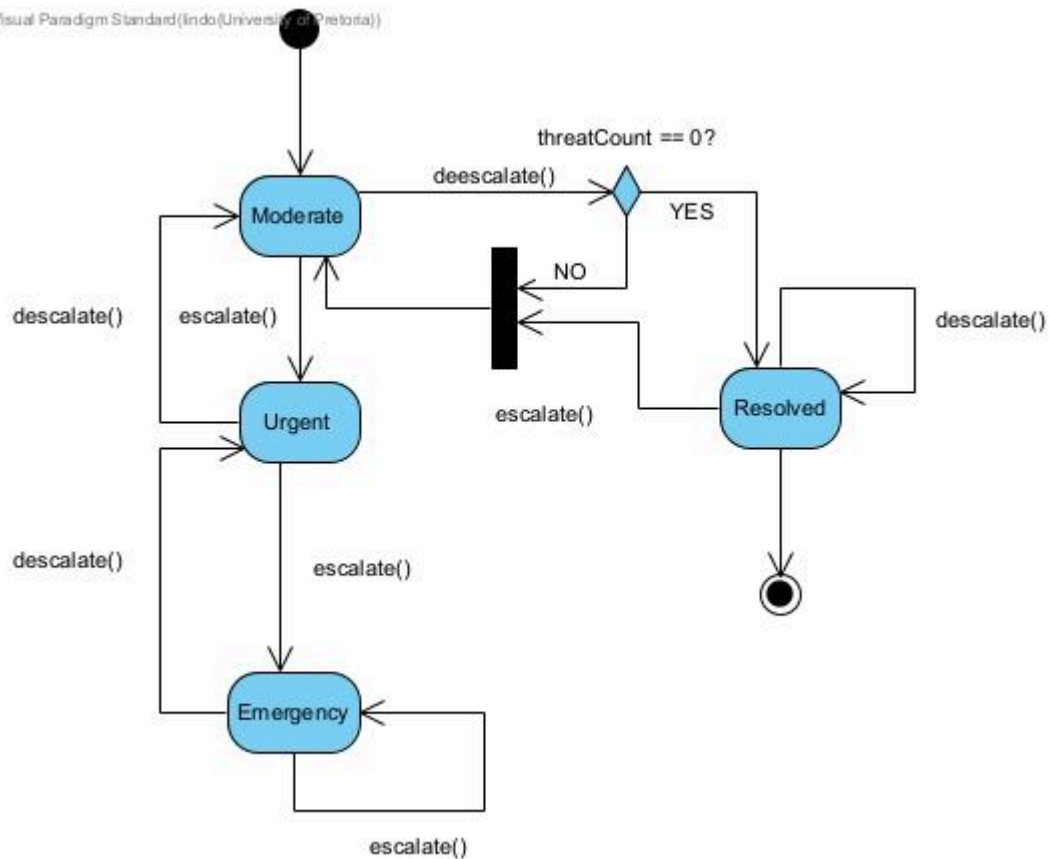


Sequence Diagram that includes Facade or Adapter



State Diagram

Visual Paradigm Standard (indo (University of Pavia))



Memory Management and Debugging

```

$ make valgrind
== STATE ==
==80085== Stack overflow in thread #1: can't grow stack to 0x1ffe801000
==80085==
==80085== Process terminating with default action of signal 11 (SIGSEGV)
==80085== Access not within mapped region at address 0x1ffe801ff0
==80085== Stack overflow in thread #1: can't grow stack to 0x1ffe801000
==80085== at 0x4846f62: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==80085== If you believe this happened as a result of a stack
==80085== overflow in your program's main thread (unlikely but
==80085== possible), you can try to increase the size of the
==80085== main thread stack using the --main-stacksize= flag.
==80085== The main thread stack size used in this run was 8388608.
==80085==
==80085== HEAP SUMMARY:
==80085==   in use at exit: 5,460,992 bytes in 74,811 blocks
==80085== total heap usage: 74,919 allocs, 108 frees, 5,472,906 bytes allocated
==80085==

```

```

== STATE ==
Program received signal SIGSEGV, Segmentation fault.
0x00007ffff78ac5b9 in _int_malloc (av=av@entry=0x7ffff7a03ac0 <main_arena>, bytes=72) at ./malloc/malloc.c:4460
warning: 4460 ./malloc/malloc.c: No such file or directory
(gdb) backtrace
#0 0x00007ffff78ac5b9 in _int_malloc (av=av@entry=0x7ffff7a03ac0 <main_arena>, bytes=72) at ./malloc/malloc.c:4460
#1 0x00007ffff78ad8f2 in _GI_libc_malloc (bytes=<optimized out>) at ./malloc/malloc.c:3328
#2 0x00007ffff7cbb904 in operator new(unsigned long) () from /lib/x86_64-linux-gnu/libstdc++.so.6
#3 0x000055555555619b in IncidentControl::IncidentControl (this=0x555555b3b208) at IncidentControl.cpp:11
#4 0x00005555555562183 in Moderate::Moderate (this=0x555555b3b200) at /home/vashti/COS214/COS214_Prac5/Moderate.h:7
#5 0x000055555555619da in IncidentControl::IncidentControl (this=0x555555b3b1b8) at IncidentControl.cpp:11
#6 0x00005555555562183 in Moderate::Moderate (this=0x555555b3b1b0) at /home/vashti/COS214/COS214_Prac5/Moderate.h:7
#7 0x000055555555619da in IncidentControl::IncidentControl (this=0x555555b3b168) at IncidentControl.cpp:11
#8 0x00005555555562183 in Moderate::Moderate (this=0x555555b3b160) at /home/vashti/COS214/COS214_Prac5/Moderate.h:7
#9 0x000055555555619da in IncidentControl::IncidentControl (this=0x555555b3b118) at IncidentControl.cpp:11
#10 0x00005555555562183 in Moderate::Moderate (this=0x555555b3b110) at /home/vashti/COS214/COS214_Prac5/Moderate.h:7
#11 0x000055555555619da in IncidentControl::IncidentControl (this=0x555555b3b0c8) at IncidentControl.cpp:11
#12 0x00005555555562183 in Moderate::Moderate (this=0x555555b3b0c0) at /home/vashti/COS214/COS214_Prac5/Moderate.h:7
#13 0x000055555555619da in IncidentControl::IncidentControl (this=0x555555b3b078) at IncidentControl.cpp:11
#14 0x00005555555562183 in Moderate::Moderate (this=0x555555b3b070) at /home/vashti/COS214/COS214_Prac5/Moderate.h:7
#15 0x000055555555619da in IncidentControl::IncidentControl (this=0x555555b3b028) at IncidentControl.cpp:11
--Type <RET> for more, q to quit, c to continue without paging--

```

```

==93234==
==93234== HEAP SUMMARY:
==93234==   in use at exit: 0 bytes in 0 blocks
==93234== total heap usage: 145 allocs, 145 frees, 92,950 bytes allocated
==93234==
==93234== All heap blocks were freed -- no leaks are possible
==93234==
==93234== For lists of detected and suppressed errors, rerun with: -s
==93234== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```